BookLeaf Author Support Portal

Complete Technical Architecture, File Directory & Implementation Reference

Project: BookLeaf Support Portal	**Tech Stack:** Node, Express, TS, Mongo, React, Vite, Socket.IO
Status: 100% Complete & Verified	**Verification:** All boundary & security suites passed
Portals: Author Portal + Admin Desk	**AI Model:** Zod-validated 6-Category Triage + KB Drafts

1. Executive Project Summary (Project Kya Hai)

The **BookLeaf Author Support Portal** is a production-grade, dual-portal web application built to eliminate friction between independent authors and publishing operations. It is composed of two primary modules:

- **Author Portal:** Allows authors to inspect their catalog of books, track live sales figures, view royalty ledgers (earned, paid, and pending payouts), submit support tickets with a book selector or general account level, and converse with publishing specialists in real time. Crucially, pre-publication books in production stages display '—' for pending retail values without crashing.
- **Admin Support Desk:** Empowers staff to triage incoming tickets sorted by an urgency-ranked queue, filter across 6 official categories and 4 priority tiers, update status and assignees, review and customize AI-generated draft responses, and write private internal notes.
- **Strict AI Safety Mandate:** AI operates strictly as an assistant. All classification outputs are validated via Zod schemas into the 6 authorized categories. Response drafts are grounded exclusively in official BookLeaf Knowledge Base policy facts. **No reply is ever automatically sent to an author without explicit human admin review.**

Evaluation Area	Weight	Implementation & Compliance
AI Integration Quality	25%	Two-stage decoupled triage; Zod-validated classification (6 categories, 4 priorities); KB-grounded draft generator; 0% auto-send.
Architecture & Code Quality	25%	Modular client/server separation; TypeScript throughout; HttpOnly JWT sessions; idempotent seed script; database-level privacy filtering.
API Design	15%	RESTful conventions; standardized response envelopes; authorId derived from session (never client body); robust HTTP error codes.
Functionality & Completeness	15%	Dual portals; real-time Socket.IO synchronization; null-safe production book handling; full ticket lifecycle management.
Product Thinking	10%	1-click demo account switcher; urgency badges; private internal notes; clear separation of author vs. staff perspectives.
Documentation & Communication	10%	Exhaustive README, complete API documentation, architectural write-up, and automated verification suite.

2. File & Folder Statistics (Kitni Files Hain)

The project is structured into three distinct zones: **Root Workspace**, **Server Backend (`server/`)**, and **Client Frontend (`client/`)**. Here is the breakdown of files and folders inside the repository:

Directory / Area	File Count	Key Responsibilities
Root Directory	10 files	Docker Compose orchestration (docker-compose.yml, .env.docker.example), package scripts, git hygiene (.gitignore), API docs, architecture write-up, and complete documentation.
Server Backend (`server/`)	22 files	Dockerfile, .dockerignore, Express API, TypeScript config, MongoDB Mongoose schemas, HttpOnly auth middleware, AI triage service, Socket.IO hub, 7 pre-seeded tickets & notes, and verification suite.
Client Frontend (`client/`)	21 files	Multi-stage Dockerfile, Nginx reverse proxy configuration, React 18, Vite bundler, BookLeaf design system, 8 modular UI components, API client, and Socket.IO client.
Total Project Footprint	**53 code/doc files**	**Complete production-ready stack with multi-tier Docker containers. Excludes node_modules and dist build artifacts.**

3. Server Backend Architecture & File Reference (`server/`)

The server backend is implemented in TypeScript using Express and Mongoose. It exposes a fully authenticated REST API paired with a Socket.IO WebSocket server. Below is the detailed breakdown of every single file in `server/`:

server/src/server.ts — Application Bootstrap

Initializes the Node HTTP server, connects to MongoDB, attaches the Socket.IO instance, and binds to PORT 5000.

server/src/app.ts — Express Pipeline

Configures Express, Helmet security headers, CORS with credentials for Vite (port 5173), cookie parser, JSON body limit (1MB), route mounts, 404 handler, and centralized error handler.

server/src/config/db.ts — Database Connection

Manages the Mongoose connection lifecycle to MongoDB (local or Atlas URI), handling connection errors and graceful logging.

server/src/middleware/auth.ts — Session & RBAC

Verifies the accessToken HttpOnly JWT cookie (or Bearer header), retrieves the active user from MongoDB, attaches AuthenticatedUser to request context, and provides requireRole('admin') / requireRole('author').

server/src/middleware/validate.ts — Zod Body Validator

Generic middleware taking a Zod schema to validate request payloads before route execution. Rejects malformed requests with standard 400 VALIDATION_ERROR.

server/src/models/User.ts — User Model

Schema storing name, email (unique/lowercased), passwordHash (hidden by default via select: false), role ('author'|'admin'), authorId link, and isActive boolean.

server/src/models/Author.ts — Author Profile Model

Schema for author metadata: authorId (unique), name, email, phone, city, and joinedDate.

server/src/models/Book.ts — Book & Royalty Model

Schema tracking bookId, authorId, title, isbn, genre, publicationDate, status, mrp, authorRoyaltyPerCopy, copiesSold, royaltyEarned, royaltyPaid, royaltyPending, lastPayoutDate, printPartner, and availableOn platforms.

server/src/models/Ticket.ts — Support Ticket Model

Schema for authorId, bookId (nullable for General inquiries), subject, description, category (6 allowed enum values), categorySource ('ai'|'admin'|'default'), priority (4 tiers), prioritySource, status (5 stages), assignedAdminId, and aiSuggestionAvailable.

server/src/models/TicketMessage.ts — Ticket Message Model

Stores ticketId, senderId, senderRole ('author'|'admin'), body, and visibility ('public'|'internal'). Essential for role boundary enforcement.

server/src/models/AIMeta.ts — AI Audit Log Model

Logs AI audit records: ticketId, task ('classification'|'draft'), model name, promptVersion, confidence score, latency in ms, success flag, and failureReason.

server/src/routes/auth.routes.ts — Auth Routes

Endpoints: POST /api/auth/login (verifies bcrypt password, sets HttpOnly cookie), POST /api/auth/logout (clears cookie), and GET /api/auth/me (returns current authenticated session).

server/src/routes/books.routes.ts — Book Routes

Endpoint: GET /api/books. For authors, automatically scopes query to authorId (Author A cannot see Author B's books). For admin, returns all 18 catalog books.

server/src/routes/tickets.routes.ts — Author Ticket Routes

POST /api/tickets (saves ticket, extracts authorId from session, executes AI triage, emits socket), GET /api/tickets (lists author tickets), GET /api/tickets/:id (strictly excludes internal notes for authors), POST /api/tickets/:id/messages (public author reply).

server/src/routes/admin.routes.ts — Admin Support Routes

GET /api/admin/tickets (queue with urgency sort & filters), PATCH /api/admin/tickets/:id (triage updates), POST /api/admin/tickets/:id/notes (private internal notes), POST /api/admin/tickets/:id/draft-response (AI draft), POST /api/admin/tickets/:id/send-response (explicit admin send).

server/src/services/ai.service.ts — AI Triage & Draft Service

Implements classifyTicket and draftReply. Uses OpenAI API with JSON mode when OPENAI_API_KEY is configured. Includes a deterministic, Zod-validated heuristic engine for offline/demo resilience.

server/src/services/knowledgeBase.ts — Publishing Policy Engine

Source-of-truth knowledge facts across all 6 categories (80/20 profit split, quarterly payouts, Rs. 1,000 minimum balance, 5-7 day print turnaround, etc.). Feeds exact facts to AI drafting.

server/src/sockets/index.ts — Real-Time WebSocket Hub

Socket.IO server attaching to Node HTTP server. Authenticates connection via JWT. Manages admin-room and author: rooms. Emits ticket:created, ticket:updated, and message:created (strictly gating internal notes to admin-room).

server/scripts/seed.ts — Idempotent Database Seeder

Parses server/data/sample.json, upserts admin account, 10 authors, 18 books, 7 realistic tickets across all 6 categories, public messages, private internal admin notes, and AI classification audit records. Idempotent and safe.

server/scripts/verify-all.ts — End-to-End Test Suite

Automated verification test script testing all 6 core requirements: API health, auth, author book boundary isolation, AI triage, internal note redaction, and staff review.

server/Dockerfile — Production Node.js 22 Container

Multi-layer cached production image with native C++ build tools for bcrypt, automated TypeScript compilation, and Node.js 22 runtime on port 5000.

server/.dockerignore — Build Context Exclusions

Prevents local node_modules, dist, and environment secrets from being copied into the server container.

4. Client Frontend Architecture & File Reference (`client/`)

The frontend is built with React 18, Vite, and TypeScript. It features a custom BookLeaf design system with no external generic styling dependencies, using pure CSS variables, crisp typography, and responsive grid layouts:

client/src/App.tsx — Root Coordinator & Router

Main application shell. Maintains active user state, manages portal view switching (Author Portal tabs vs. Admin Desk tabs), subscribes to real-time Socket.IO events, and synchronizes ticket counters.

client/src/index.css — Design System & Tokens

Comprehensive CSS tokens: BookLeaf deep forest green (#0f3d2e), emerald (#10b981), priority badges (Critical, High, Medium, Low), status indicators, animations (fadeIn, pulseGlow), custom scrollbars, and internal note dashed callouts.

client/src/types/index.ts — TypeScript Schema Interfaces

Shared TypeScript types: User, Book, Ticket, TicketCategory, TicketPriority, TicketStatus, TicketMessage, and TicketDetailResponse.

client/src/services/api.ts — REST API Client

Type-safe fetch client handling credentials: 'include' for HttpOnly cookies. Exposes clean helper functions for all auth, book, author ticket, and admin triage endpoints.

client/src/services/socket.ts — WebSocket Client

Singleton Socket.IO client instance. Manages auto-reconnect, joinTicketRoom, leaveTicketRoom, joinAdminRoom, and joinAuthorRoom subscriptions.

client/src/components/Navbar.tsx — Global Header Bar

Displays BookLeaf logo, portal badge ('Author Portal' vs 'Admin Desk'), live sync status indicator dot, quick demo account switcher trigger button, user info, and logout.

client/src/components/LoginModal.tsx — 1-Click Demo & Auth Modal

Authentication modal featuring 1-click quick-login buttons for BookLeaf Admin, Priya Sharma (AUTH001), Sneha Kulkarni (AUTH007 - production book), and Rohit Kapoor (AUTH002), plus manual email/password input.

client/src/components/AuthorBooks.tsx — My Books & Royalty Ledger

Displays 4 top-level aggregate stat cards (Total Books, Copies Sold, Royalty Earned, Pending Payout) and a card grid of books. Implements strict null-safe handling (renderValueOrDash) displaying '—' for in-production titles.

client/src/components/AuthorNewTicket.tsx — Support Ticket Submission Form

Form with book dropdown (including 'General / Account Level' option), subject input, and detailed description. Triggers instant AI triage and provides real-time category/priority confirmation.

client/src/components/AuthorTickets.tsx — Author Ticket List & Thread

Displays author's tickets with status tags. Clicking opens the full message conversation. Staff replies appear in green cards. Internal notes are strictly filtered out at the server layer.

client/src/components/AdminQueue.tsx — Triage Queue with Urgency

Admin queue showing tickets with pulsing red badges for Critical urgency. Features search bar, filters for Status, Category (6), Priority (4), and Sort (Urgent First vs FIFO Oldest).

client/src/components/AdminTicketDetail.tsx — Staff Triage & AI Reply Workspace

Comprehensive triage workspace: dropdowns to reclassify Category, Priority, Status, and Assignee; 'Generate AI Draft Reply' button; editable compose reply box with explicit send; and private internal note creator.

client/src/components/AdminBooksView.tsx — Complete Catalog Database

Admin view of all 18 titles across all 10 authors with aggregate catalog statistics, status filters, and search functionality.

client/vite.config.ts — Vite Dev Server & Proxy

Configures local dev server on port 5173 with automatic reverse-proxying of /api and /socket.io requests to backend at http://localhost:5000.

client/index.html — HTML5 Entrypoint

Loads Google Fonts (Plus Jakarta Sans, Outfit, JetBrains Mono), meta tags, custom BookLeaf SVG favicon, and mounts the React root.

client/Dockerfile — Multi-Stage Nginx Runner

Stage 1 compiles the optimized production React bundle; Stage 2 serves assets through lightweight Nginx Alpine on port 80.

client/nginx.conf — Reverse Proxy Configuration

Routes /api and /socket.io (WebSocket upgrade) to server:5000 and provides client-side SPA routing fallback.

client/.dockerignore — Build Context Exclusions

Excludes local node_modules and dist artifacts from the Docker build context.

5. Step-by-Step Workflow & Data Flow (Kaise Kaam Karta Hai)

Here is the complete chronological path of data through the system from ticket creation to resolution:

Step 1: Author Login & Isolation

Author signs in (e.g., Priya Sharma, AUTH001). The backend issues an HttpOnly JWT cookie. When the frontend requests GET /api/books, the server automatically reads the session authorId and queries only books matching that ID. Author A never sees Author B's records.

Step 2: Viewing Royalties & Null-Safety

The author reviews the live ledger. If the author owns an in-production book (such as Sneha Kulkarni owning BK013 - 'Midnight in Mysore'), fields like publication date, MRP, and royalty rate return null from MongoDB. The frontend renders '—' rather than crashing.

Step 3: Submitting Support Ticket

Author fills the ticket form, choosing a specific book or 'General / Account Level'. The payload contains only { bookId, subject, description }. The server injects authorId directly from the verified session, protecting against client-side spoofing.

Step 4: Persistence-First AI Classification

The server writes the ticket to MongoDB first with default status 'Open'. It then invokes classifyTicket in ai.service.ts. The AI output is parsed via Zod into one of the 6 allowed categories and 4 priority tiers. The ticket is updated and an AIMeta audit record is stored.

Step 5: Real-Time Broadcast

The server triggers emitTicketCreated via Socket.IO. The admin support desk receives the event and immediately updates the triage queue without requiring a page refresh.

Step 6: Admin Triage & Urgent Ranking

The admin sees the ticket at the top of the queue with an urgency indicator. The admin can click 'Triage & Respond' to re-assign category, priority, or status.

Step 7: Knowledge Base AI Reply Drafting

The admin clicks 'Generate AI Draft Reply'. The backend loads relevant policy facts from knowledgeBase.ts (e.g. quarterly payment timing, Rs. 1,000 minimum balance) and drafts an empathetic, factual reply. The draft is placed in an editable textarea.

Step 8: Human-in-the-Loop Staff Review

The admin reviews, refines, or edits the draft. When ready, the admin clicks 'Send Public Reply'. Auto-send is structurally prohibited, guaranteeing zero hallucinated messages reach authors.

Step 9: Adding Private Internal Notes

Staff members can record private internal notes (e.g. bank transaction reference, printer batch status). These notes are flagged with visibility: 'internal' and are strictly filtered out on author-facing endpoints.

Step 10: Author Real-Time Notification

When the staff reply is submitted, Socket.IO broadcasts message:created to the author's room. The author's conversation view updates instantly with the verified response.

6. Demo Accounts & Verification Guide

The system includes pre-seeded demo accounts accessible via the 1-click account switcher in the top navigation bar:

Role	Name / ID	Email	Password	Key Features to Test
Admin	BookLeaf Admin	`admin@bookleaf.com`	`AdminPassword123!`	Full triage queue, AI draft generator, private notes, status controls.
Author	Priya Sharma (AUTH001)	`priya.sharma@email.com`	`AuthorPassword123!`	2 published fiction & memoir titles, royalty tracking, ticket creation.
Author	Sneha Kulkarni (AUTH007)	`sneha.kulkarni@email.com`	`AuthorPassword123!`	3 books: 2 published + 1 in production (BK013, tests null-safety '—').
Author	Rohit Kapoor (AUTH002)	`rohit.kapoor@email.com`	`AuthorPassword123!`	Best seller (2,000+ copies sold, Rs. 84k+ royalty earnings).
Author	Kavita Deshmukh (AUTH009)	`kavita.deshmukh@email.com`	`AuthorPassword123!`	2 books: 1 published + 1 in typesetting pipeline (BK015).

Docker Compose (Production-Grade Stack):

- **Start All Containers:** docker compose up -d (or npm run docker:up)
- **Seed DB in Container:** docker compose exec server npm run seed (or npm run docker:seed)
- **Follow Live Logs:** docker compose logs -f (or npm run docker:logs)
- **Stop All Containers:** docker compose down (or npm run docker:down)

Local Node.js Development (Alternative):

- **Run Seeder:** cd server && npm run seed (Idempotent upsert of 10 authors, 18 books, 7 tickets, 1 admin)
- **Run Verification Suite:** cd server && npx tsx scripts/verify-all.ts (Automated 6-point test suite)
- **Start API Server:** cd server && npm run dev (Runs Express + Socket.IO on port 5000)
- **Start Frontend Client:** cd client && npm run dev (Runs Vite React app on port 5173)